Working with data in Python 1

Python Programming for Business Intelligence

Sippo Rossi
March 25th 2026

Learning goals

Theoretical:

- Understand what modules, packages and libraries are and why we use them
- Know what NumPy and Pandas are
- Know when to use NumPy and when to use Pandas

Practical:

- Know how the basic features of NumPy and Pandas
- Know how to find more information about NumPy and Pandas via documentation

Motivation

While you can use the base Python for many things, in practice we use external packages most of the time

This session will introduce you to two packages, NumPy and Pandas that are widely used when working with data in Python

Even if you do not end up using NumPy or Pandas beyond this course, their syntax and design principles are followed by many other packages and libraries

Assignments and exercises go beyond the lecture materials

Similar to Excel, with its thousands of features most do not even know about, NumPy and Pandas have very many features

The exercises and assignments sometimes requires you to search for features not covered during the lectures

This is intentional, as it is an important skill in programming to be able to search for information about the tools that you use and to discover whether a readymade solution exists

Agenda

- Modules, packages, libraries
- NumPy
- Pandas

Modules, libraries and packages

Modules

To make managing large programs easier, software is typically divided into smaller pieces (separate files)

In Python these pieces are called *modules*

Modules can contain classes, object functions, among other things

Python has hundreds of built in modules

You can create your own modules simply by saving code in a .py file. You can then import components from this file onto other files or your Jupyter Notebook

Slide adapted from University of Helsinki's Data Analysis with Python

Packages and libraries

Packages are a collection of modules. Sometimes people use library and package interchangeably, although to be precise **a library is a collection of packages**.

These contain typically functions, classes and methods for performing tasks such as data manipulation, mathematical operations, statistical analysis and more

The Python Standard Library has many built-in modules, and commonly you will especially in data analysis use external libraries that need to be installed

External libraries and packages can be anything from professionally developed and maintained toolkits to passion projects maintained by a hobbyist

Module, package and library demo

Going through *L2_modules_packages_libraries.ipynb*

A note about libraries

Some libraries have created standards, and many other libraries are built on top of them

Examples include NumPy, Pandas, SciPy, Scikit-learn

For any given problem there are possibly multiple suitable libraries. The popular ones have better resources and documentation

But sometimes niche libraries can be better suited for a task. E.g., as an alternative for Pandas, the Polars library can be significantly faster when working with massive datasets

A note about online resources and generative AI

Nowadays writing scripts for data analysis is easier than ever before thanks to the number of tutorials and ChatGPT

There are many resources (stackoverflow, w3schools, Medium) of varying quality available, although these are rapidly becoming obsolete due to generative AI and AI chatbots

Overall, remembering syntax is less valuable than it used to. However, some commands are good to learn by heart and while studying you learn mainly by doing. Thus, during this course, learn the concepts by doing them by hand, but of course memorizing as much syntax as possible no longer makes sense when AI exists

Also, an important skill for any programmer is the ability to search for information with Google as well as to be able to read and understand official documentation, especially when working with more esoteric libraries that AI chatbots might struggle with

NumPy and Pandas

NumPy and Pandas

NumPy arrays and Pandas DataFrames are some of the most commonly used Python data formats for data scientists

Many other libraries are built to use these data structures such as Matplotlib, Seaborn, scikit-learn among others

Why do we use these and instead of just the built-in libraries of Python?

Libraries vs base Python

Python has many built in data structures (e.g. lists and dictionaries)

The basic data structures in Python were not designed for handling large arrays of data in a convenient format for data analysis and visualization

Moreover, Python's own data structures are not efficient, as the programming language wasn't designed to optimize computational efficiency

Thus, when using Python for data science, we use libraries that have made working with data much more straightforward

NumPy

NumPy (Numerical Python)

NumPy is a Python package for working with arrays of data very efficiently

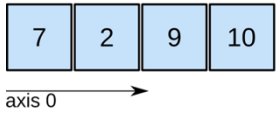
Especially compared to lists in Python, the NumPy array object is significantly faster

NumPy arrays can only have only one data type at a time

Documentation with instructions and a user guide:
https://numpy.org/doc/stable/index.html

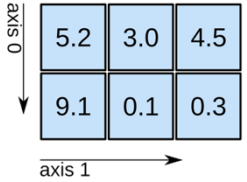
NumPy arrays

1D array



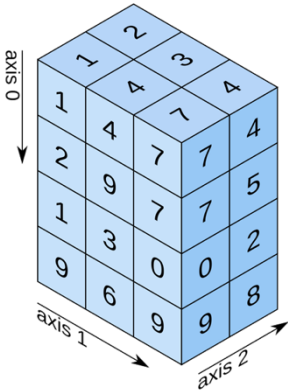
shape: (4,)

2D array



shape: (2, 3)

3D array



shape: (4, 3, 2)

```
list = [1,2,3]
list
```

```
[1, 2, 3]
```

```
import numpy as np
arr = np.array([1,2,3])
arr
```

```
array([1, 2, 3])
```

NumPy demo

Going through *L2_numpy.ipynb*

When does using NumPy make sense?

NumPy can be used only when working with arrays consisting of one type of data (e.g., floats)

In situations where you have multiple columns of different data types in your dataset you should use Pandas instead

Pandas

Pandas

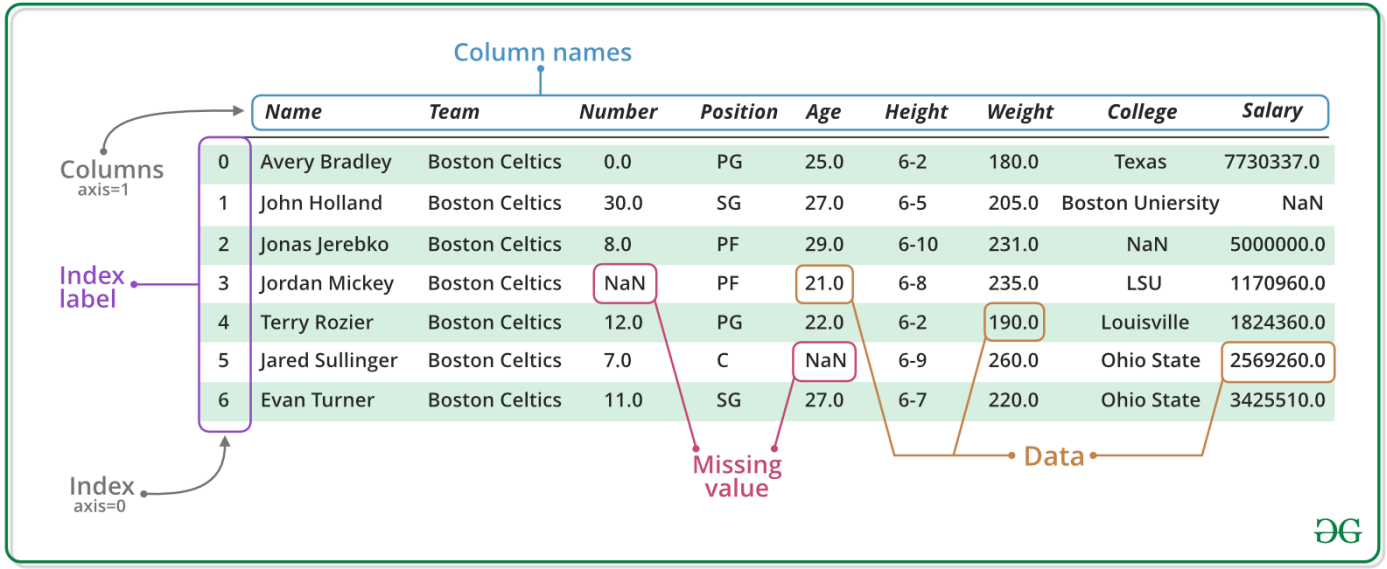
Built on top of NumPy, Pandas is the other standard package for data analysis in Python

The primary data structure within Pandas is the DataFrame, which supports two-dimensional tabular data

The most notable difference to a NumPY array is that Pandas' DataFrames can have multiple different data types in different columns

Documentation with instructions and a user guide:
https://pandas.pydata.org/docs/

Pandas DataFrames



	Name	Team	Number	Position	Age	Height	Weight	College	Salary
0	Avery Bradley	Boston Celtics	0.0	PG	25.0	6-2	180.0	Texas	7730337.0
1	John Holland	Boston Celtics	30.0	SG	27.0	6-5	205.0	Boston Uniersity	NaN
2	Jonas Jerebko	Boston Celtics	8.0	PF	29.0	6-10	231.0	NaN	5000000.0
3	Jordan Mickey	Boston Celtics	NaN	PF	21.0	6-8	235.0	LSU	1170960.0
4	Terry Rozier	Boston Celtics	12.0	PG	22.0	6-2	190.0	Louisville	1824360.0
5	Jared Sullinger	Boston Celtics	7.0	C	NaN	6-9	260.0	Ohio State	2569260.0
6	Evan Turner	Boston Celtics	11.0	SG	27.0	6-7	220.0	Ohio State	3425510.0

Image from GeeksForGeeks

Pandas demo

Going through *L2_pandas.ipynb*

Selecting rows and columns

Columns and rows can be selected in several ways with Pandas

Rows and columns can be accessed e.g. by index number, with a boolean array, or by label (row or column name)

In the following examples, the dataset has been stored as a DataFrame named "df"

Using .loc[]

Used to access rows or columns by their labels or with a boolean array

Commonly row labels are numbers. Note that .loc[1] will be interpreted as the label of the index, not the position

When using slices, unlike with typical python slicing, both the start and stop are included

Read more here

Using .loc[]

df.loc[1] will return the row with label 1 as a series

df.loc[[1]] will return the row with label 1 as a DataFrame

df.loc[0:5] will return all rows from start (0) to the end (5)

df.loc[:,"column_name"] will return all rows and the named column

See the next Jupyter notebook for more examples

Using .iloc[]

Used to access rows or columns by their integer position or with a boolean array

The positions go from 0 to length – 1

Note that when using slices the end number is not included!

For selecting rows, .iloc[] is faster than .loc[]

Read more here

Using .iloc[]

df.iloc[1] will return the row at position 1 as a series

df.iloc[[1]] will return the row at position 1 as a DataFrame

df.iloc[0:5] will return all rows from index (0) to (4)

df.iloc[:, 0:2] will return all rows and the selected columns

See the next Jupyter notebook for more examples

Selecting data without .iloc[] or .loc[]

DataFrames can be subsetted without .iloc or .loc simply like this:

df[0:5] to get rows

df["column_name"] or **df[["col_name1", col_name2"]]** to get columns

df[0:5][["col_name1", col_name2"]] to get a subset of rows and columns

However, it is preferable to use .iloc or .loc as they are explicit and clear, especially when it comes to accessing data using integer indexes

Data selection demo

Going through *L2_data_selection.ipynb*

Subsetting DataFrames and referencing

Note that there is an important distinction between ***copying*** object and ***referencing*** objects

When subsetting datasets and storing the results in a new DataFrame the new one is still referencing the original DataFrame

Use the Pandas method .copy() to create a new true copy of a DataFrame

Why does this matter? If you change some value in the subset that is referencing the original df, the data gets changes in both!

Conclusion

Conclusion

Data analysis in Python is mostly done using packages such as NumPy and Pandas among others

During the exercise session you will get more experience in working with them

Note that the first assignment is due on Sunday this week!

Questions, comments suggestions?